

Machine Learning Final Project Report

Telco Customer Churn Prediction

Dana Bolden | ITAI 1371 – 12321

1. Problem Framing

Acquiring a new telecom customer costs far more than retaining an existing one, which is what makes churn prediction worth solving. This project treats churn as a binary classification problem: using account engagement, service adoption, and billing history, the goal is to flag which customers are likely to leave (1) and which are likely to stay (0). A model that gets this right lets the business step in before a customer cancels, rather than after, targeting retention offers at the accounts actually at risk instead of spreading them across the whole customer base.

2. Data Summary & Preprocessing Recap

The project draws on the Telco Customer Churn dataset: 3,333 customer records across 32 variables. Before any modeling started, a data integrity audit turned up a quiet but serious problem: the TotalRevenue column held blank spaces for a subset of customers, which forced it to be read as text rather than as numbers. Left uncorrected, that single formatting issue would have pushed the feature matrix past 3,000 columns once One-Hot Encoding ran.

The fix: coerce TotalRevenue to a numeric type, impute the resulting nulls with the column median, map Churn to a clean 0/1 target, and One-Hot Encode the remaining categorical variables with drop_first=True to avoid the dummy variable trap.

3. Feature Engineering

The midterm feedback on this project was blunt about the feature engineering: too much reliance on simple unit conversions, such as dividing tenure months by twelve. That approach added noise more than insight. The rework instead builds compound behavioral features, signals that describe what a customer actually does, rather than just how long they have held an account. Two features carry most of that weight:

- **Total_AddOn_Services** – a running count of the ecosystem features a customer has active: Online Security, Online Backup, Device Protection, Tech Support, Streaming TV, and Streaming Movies. The more of these a customer uses, the more embedded they are in the provider's infrastructure, and the higher the switching cost of leaving.
- **Total_Calls_All_Types** – Day, Evening, Night, and International call volumes rolled into one number that captures overall network usage instead of four separate, noisier ones.

4. Methodology

Data leakage was the main risk to guard against, so the split came first: 70/15/15 for training, validation, and test, with a fixed random seed (42) for reproducibility and stratification to keep the roughly 14% churn rate consistent across all three sets.

The training set alone received the Synthetic Minority Over-sampling Technique (SMOTE), which brought the class balance to 50/50, along with standard feature scaling (StandardScaler). Both were fit only on that 70% training split; validation and test were kept untouched by the balancing process, so the results reported below reflect real-world class distribution rather than an inflated synthetic one.

5. Full Results Table & Best-Model Analysis

Six algorithms were trained on the SMOTE-balanced data: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, K-Nearest Neighbors (KNN), and a Support Vector Classifier (SVC). Random Forest received a light GridSearchCV pass over `n_estimators` and `max_depth`, mainly to push it past default-parameter performance before comparing it against the others.

Results below are against the 15% validation set, unseen during training:

Model	Val Accuracy	Test Accuracy	Val F1	Test F1	Val ROC-AUC	Test ROC-AUC
Gradient Boosting	0.936	0.928	0.789	0.757	0.973	0.955
Random Forest (Tuned)	0.906	0.910	0.662	0.667	0.937	0.929
SVC	0.892	0.902	0.667	0.671	0.933	0.927
Logistic Regression	0.828	0.846	0.587	0.597	0.917	0.912
KNN	0.792	0.796	0.552	0.536	0.863	0.858
Decision Tree	0.880	0.890	0.647	0.636	0.828	0.797

Best-Model Justification: ROC-AUC is the metric that matters most here; it weighs the true-positive/false-positive trade-off directly, and in imbalanced classification, missing a customer who is about to churn is an expensive mistake. By that standard, Gradient Boosting won outright (Val AUC: 0.973), with the tuned Random Forest a clear second (Val AUC: 0.937). The gap makes sense: boosting builds trees sequentially, each one correcting the last one's errors, which suits the non-linear behavioral patterns in this data better than a single tree or a bagged forest can match, and without the overfitting a deeper single tree would show.

6. Ensemble Analysis

The top three validation performers, Gradient Boosting, Random Forest, and SVC, were combined into a Soft Voting Classifier to test whether averaging their predicted probabilities would beat any single model. A Gaussian Naive Bayes model ran alongside it as a simple Bayesian baseline, mainly to confirm the ensemble was earning its added complexity.

Final results, against the test set, none of the models had seen at any stage:

Ensemble Model	Test Accuracy	Test F1	Test ROC-AUC
Soft Voting Ensemble	0.918	0.705	0.945
GaussianNB Baseline	0.690	0.448	0.892

The ensemble beat the Naive Bayes baseline by a wide margin, which was expected. More interesting is that standalone Gradient Boosting edged out the ensemble on raw test numbers (0.955 vs. 0.945 ROC-AUC), for a single deployed model, which would argue for just shipping Gradient Boosting. But the Voting Classifier spreads risk across three different model families instead of betting everything on one, and in production, that resilience against any single model's blind spots is often worth more than half a point of AUC.

7. Limitations

Two limitations are worth flagging. SMOTE builds its synthetic minority samples by interpolating between real ones, and in a high-dimensional feature space, those interpolated points can miss the odder, edge-case churn behaviors that do not sit neatly between two existing customers. And hyperparameter tuning went only as far as a small grid search on Random Forest; compute constraints ruled out anything more exhaustive. A RandomizedSearchCV pass on Gradient Boosting specifically is the most likely place to find further gains.

8. Conclusion

The biggest lesson from this project was not which algorithm won; it was how much the early decisions mattered. Catching the TotalRevenue type corruption before it silently broke the feature matrix, replacing a weak unit-conversion feature with real behavioral signals, and keeping SMOTE strictly inside the training fold were what made the final numbers trustworthy rather than just high. The Soft Voting Ensemble's 0.945 ROC-AUC on held-out test data reflects a model that generalizes rather than one that memorizes its training set, and that is the actual bar for any churn model that gets deployed.